

Del Volumen al 2.5D: Segmentación de Tumores Pulmonares con Vision Transformers Slice-wise

Análisis de Encoders 2D Pre-entrenados y Robustez ante la Escasez de Datos

Francisco Javier Mercader Martínez Chaimae Chahlal Hiba Chakour

5 de mayo de 2026

Resumen

En este proyecto hemos estudiado la segmentación automática de tumores pulmonares en tomografías computarizadas (CT) usando un enfoque **2.5D slice-wise**. Partimos del dataset MSD Task06_Lung (Antonelli et al. 2022; Simpson et al. 2019) y construimos un flujo reproducible con MONAI (Project MONAI 2020), PyTorch, Hydra y `timm`. El modelo desarrollado, **ViT25D**, combina un encoder **Vision Transformer** (Dosovitskiy et al. 2021) pre-entrenado con **Masked Autoencoders** (He et al. 2022), un decoder convolucional 2D y supervisión profunda. Para aportar contexto volumétrico sin usar una red 3D completa, cada corte axial se segmenta junto a cortes vecinos, tres ventanas HU y una máscara pulmonar pre-computada con `lungmask` (Hofmanninger et al. 2020). También implementamos una pérdida **Focal Tversky** (Abraham y Khan 2019), validación por volumen completo, inferencia por ventana deslizante y un estudio de ablación con distintas fracciones de datos. Los resultados obtenidos son modestos, en parte por las restricciones de hardware y tiempo de entrenamiento, pero el trabajo deja una base técnica y experimental coherente sobre la que se podría seguir mejorando.

Tabla de contenidos

1. Introducción	2
2. Alcance y Evolución del Proyecto	3
3. Datos	3
3.1. MSD Task06_Lung	3
3.2. Particiones y Prevención de Fugas	3
4. Preprocesamiento	4
4.1. Normalización Espacial	4
4.2. Máscara Pulmonar como Prior Anatómico	4
4.3. Ventanas HU Multicanal	4
4.4. Muestreo y Aumentación	5
5. Arquitectura ViT25D	6
5.1. Idea General	6
5.2. Flujo de Tensores	7
5.3. Encoder ViT Pre-entrenado	7
5.4. Decoder y Supervisión Profunda	8
5.5. Congelación y Descongelación	8
6. Función de Pérdida	9

7. Entrenamiento	10
7.1. Configuración Base	10
7.2. Validación por Volumen Completo	10
7.3. Checkpoints Ligeros	11
8. Inferencia Volumétrica	11
9. Fase 6: Ablación	13
10. Resultados Cualitativos	14
10.1. Curvas de Entrenamiento	14
10.2. Visualización Triplanar	14
10.3. Mosaico Axial	15
11. Ingeniería y Reproducibilidad	16
11.1. Estructura del Paquete	16
11.2. Comandos Principales	16
11.3. Tests	16
12. Discusión	17
13. Conclusiones	17
13.1. Validez del Proyecto Bajo Restricciones de Hardware	18
14. Bibliografía	18

1. Introducción

La segmentación de tumores pulmonares en CT es una tarea importante en diagnóstico asistido por ordenador, planificación radioterápica y seguimiento de respuesta a tratamiento. En la práctica clínica, delimitar una lesión requiere revisar un volumen completo corte a corte, decidir dónde empieza y termina el tumor y separar tejido tumoral de estructuras anatómicas con intensidades parecidas. Este trabajo manual es lento, depende de la experiencia del especialista y puede introducir variabilidad inter-observador.

Desde el punto de vista computacional, la segmentación médica suele formularse como un problema 3D: la entrada es un volumen (H, W, D) y la salida es una máscara de la misma geometría. La solución natural sería usar redes 3D como UNet3D, VNet, SegResNet o variantes transformer volumétricas. Sin embargo, esta aproximación tiene dos obstáculos importantes:

1. **Coste de memoria:** las convoluciones y atenciones 3D escalan de forma muy agresiva con la resolución espacial. Un parche volumétrico suficientemente grande para ver el tumor completo puede saturar con facilidad una GPU de 8 GB.
2. **Escasez de datos anotados:** MSD Task06_Lung tiene 63 volúmenes con etiqueta de entrenamiento y 32 casos de test sin máscara pública. Es una escala razonable para experimentar, pero pequeña para entrenar desde cero arquitecturas 3D profundas.
3. **Transferencia limitada en 3D:** los grandes modelos pre-entrenados disponibles en visión por computador suelen ser 2D y se han entrenado sobre millones de imágenes naturales. Una red 3D médica entrenada desde cero no aprovecha directamente ese conocimiento.

El objetivo del proyecto no es sustituir una validación clínica real ni presentar un modelo listo para uso médico. Lo que buscamos es construir un flujo de trabajo razonable, reproducible y claro en sus limitaciones. Para ello nos centramos en una pregunta concreta: **¿puede una**

arquitectura 2D pre-entrenada comportarse como un segmentador volumétrico si se le da contexto local en profundidad?

2. Alcance y Evolución del Proyecto

El repositorio comenzó como un pipeline amplio de análisis de imagen médica: exploración de CT, reconstrucción simulada, registro, segmentación, radiómica, clasificación y ablación. Durante el desarrollo se observó que esa amplitud dispersaba el esfuerzo y no mejoraba necesariamente la métrica de segmentación. Por eso el proyecto se reorientó hacia una línea principal más compacta:

- **Fase 4:** segmentación avanzada de tumores pulmonares con ViT25D.
- **Fase 6:** estudio de ablación para medir robustez ante pocos datos y distintos regímenes de aumentación.

Las fases de reconstrucción y registro quedan como conocimiento de contexto, pero no forman parte del pipeline activo. La reconstrucción simulada ayuda a entender la física de CT, aunque el dataset ya proporciona volúmenes reconstruidos. El registro inter-paciente alinea anatomía global, pero no resuelve la gran variabilidad de posición y forma del tumor. La radiómica y la clasificación se mantienen como extensión opcional para LIDC-IDRI, no para Task06, porque MSD Task06_Lung no contiene etiquetas de benignidad o malignidad.

Esta decisión de alcance es importante: la memoria no pretende presentar muchos experimentos sueltos, sino un sistema centrado en segmentación. Cada componente del código actual responde a una necesidad concreta: reducir VRAM, conservar algo de contexto 3D, controlar el desbalance de clases, evitar fugas de paciente entre folds y generar resultados que se puedan revisar después.

3. Datos

3.1. MSD Task06_Lung

El dataset principal es **MSD Task06_Lung**, parte del Medical Segmentation Decathlon (Antonelli et al. 2022). Contiene CT torácicos en formato NIfTI y máscaras binarias de tumor para los casos de entrenamiento. En la copia local del proyecto, `dataset.json` declara 63 casos con etiqueta y 32 casos de test. Como las etiquetas del test oficial no están disponibles, toda la validación experimental se realiza mediante particiones internas sobre los 63 casos anotados.

Una característica relevante del dataset es su heterogeneidad. Los tumores varían mucho en tamaño, forma, localización y contacto con estructuras de mediastino o pared torácica. Además, los CT no comparten exactamente el mismo spacing ni la misma orientación espacial. Esto obliga a estandarizar geometría y a evaluar por paciente, no por corte individual.

3.2. Particiones y Prevención de Fugas

Las particiones viven en `data/splits/fold_{0..4}.json`. Se generan con un esquema **StratifiedGroupKFold** a nivel de paciente:

- El grupo es `patient_id`, por lo que un paciente nunca aparece simultáneamente en entrenamiento y validación.
- La estratificación se aproxima mediante terciles de volumen tumoral, evitando que un fold concentre solo tumores pequeños o solo masas grandes.
- Los folds locales contienen 50/13 o 51/12 casos de entrenamiento/validación, según el reparto.

Esta elección evita una fuga frecuente en segmentación médica: partir por cortes 2D en lugar de por paciente. Si se mezclan cortes del mismo volumen entre train y val, el modelo puede memorizar contexto anatómico y la métrica queda artificialmente inflada.

4. Preprocesamiento

El preprocesamiento está implementado en `src/lungseg/data/transforms.py` mediante transformaciones MONAI. El flujo es determinista hasta el muestreo de parches y está diseñado para que el modelo reciba entradas comparables entre pacientes.

4.1. Normalización Espacial

Cada volumen se carga con `LoadImage`, se transforma a formato canal-primero con `EnsureChannelFirstd`, se reorienta a RAS mediante `Orientationd` y se remuestrea con `Spacingd` a:

$$(0.79, 0.79, 1.24) \text{ mm}$$

Este spacing conserva una resolución axial razonable y hace que el contexto de cinco cortes tenga un significado físico más estable. Si el spacing variase mucho entre pacientes, cinco cortes podrían cubrir distancias anatómicas muy distintas.

4.2. Máscara Pulmonar como Prior Anatómico

Antes del ventaneo HU se aplica una máscara pulmonar pre-computada con `lungmask` R231. La CLI incluye el subcomando:

```
python -m lungseg.cli precompute-lung-masks
```

Cada máscara se guarda en `data/cache/lung_masks/<patient_id>.nii.gz`. Durante el entrenamiento, `MaskNonLungVoxelsd` remuestrea la máscara al grid del volumen y reemplaza los vóxeles fuera de pulmón por `-1024 HU`. El objetivo es convertir el pulmón en un prior duro: el modelo no debería gastar capacidad aprendiendo que regiones claramente extra-pulmonares no son tumor.

Esta decisión también mejora la estabilidad del `CropForegroundd`. Al rellenar el exterior con aire, el recorte posterior puede centrarse en la región torácica relevante sin expandir innecesariamente el volumen.

4.3. Ventanas HU Multicanal

En CT no existe una única normalización válida para todos los tejidos. Por eso se reemplaza el canal HU original por tres ventanas clínicas, cada una normalizada a `[0, 1]`:

Canal	Rango HU	Motivación
Pulmón	<code>[-1000, 0]</code>	Parénquima, aire, bordes lesión-pulmón
Mediastino	<code>[-150, 250]</code>	Tejidos blandos y tumor sólido
Completa	<code>[-1024, 400]</code>	Contexto anatómico global

La transformación `MultiWindowHUd` se aplica después del muestreo de parches en entrenamiento. Este detalle es práctico pero importante: si se guardasen tres canales completos en `CacheDataset`, la memoria RAM se triplicaría. En el diseño actual se cachea un solo canal HU y se crean las

tres ventanas únicamente cuando el sample ya está recortado o cuando se procesa un volumen de validación.

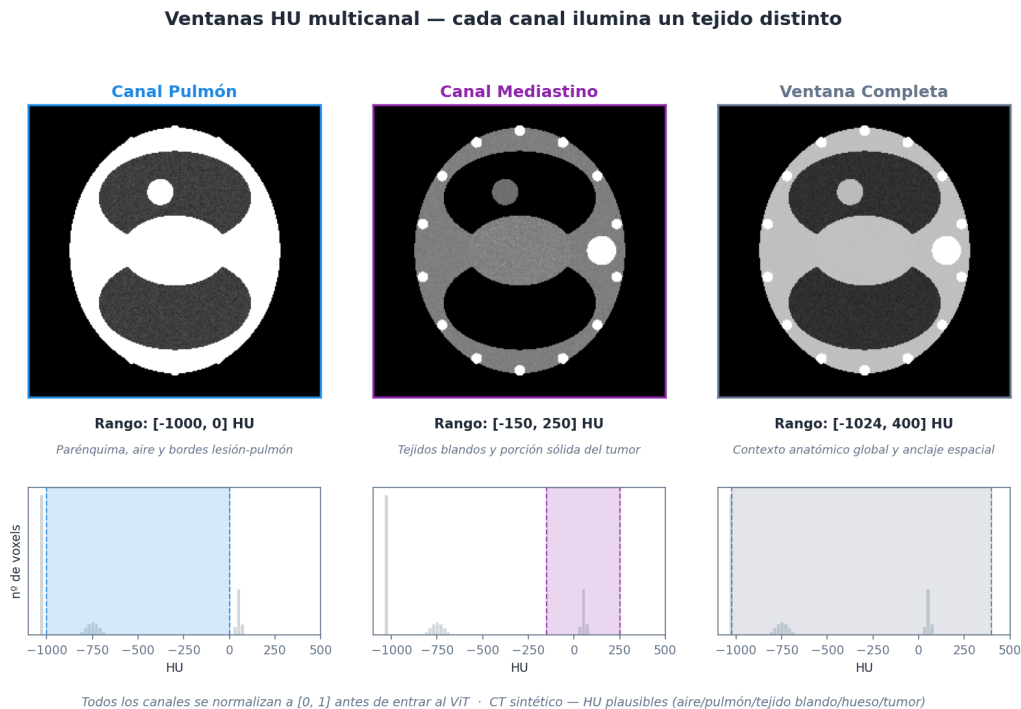


Figura 1: Las tres ventanas HU aplicadas al mismo corte axial. Cada normalización destaca un tejido distinto: pulmón $[-1000, 0]$ HU realza parénquima y bordes lesión-pulmón, mediastino $[-150, 250]$ HU muestra tumor sólido y tejidos blandos, y la ventana completa $[-1024, 400]$ HU preserva el contexto anatómico global. Los histogramas inferiores indican qué franja de la distribución HU selecciona cada ventana.

Como ilustra la Figura 1, ninguna ventana captura por sí sola toda la información clínicamente relevante: el contraste entre lesión y parénquima vive en una franja muy distinta a la del tumor sólido frente a tejidos blandos. Apilarlas como tres canales le permite al ViT atender simultáneamente a las tres escalas, sin obligar al modelo a aprender por su cuenta una compresión dinámica del rango HU.

4.4. Muestreo y Aumentación

El entrenamiento usa `RandCropByPosNegLabeld` con parches de $96 \times 96 \times 16$ en el perfil local. El sampler favorece parches positivos con `ratio pos:neg = 3:1`, porque la lesión ocupa una fracción mínima del volumen total. Sin este sesgo, muchos lotes no contendrían tumor y el gradiente estaría dominado por fondo.

El régimen de aumentación `standard` incluye ruido gaussiano, suavizado, cambios de escala/intensidad, rotaciones de 90 grados cuando la geometría lo permite y flip solo en el eje Z. El código incluye una guardia defensiva que prohíbe `RandFlipd(spatial_axis=0)` en CT de tórax: invertir izquierda-derecha puede romper convenciones anatómicas y generar ejemplos clínicamente ambiguos.

5. Arquitectura ViT25D

5.1. Idea General

ViT25D reformula la segmentación volumétrica como una secuencia de problemas 2D con contexto local en profundidad. La idea no es nueva en sí misma, pero nos permite aprovechar encoders 2D pre-entrenados y adaptarlos a un problema 3D. La entrada al modelo tiene forma:

$$(B, C, H, W, D)$$

donde $C = 3$ corresponde a las ventanas HU. Para cada corte axial z , el modelo construye un tensor 2D apilando vecinos:

$$[z - K, \dots, z, \dots, z + K]$$

Con `neighbor_context = 2`, cada predicción usa cinco cortes. Como cada corte tiene tres ventanas HU, el encoder recibe:

$$3 \times (2K + 1) = 15 \text{ canales}$$

En los bordes del volumen se usa padding por replicación: el primer corte reutiliza vecinos inexistentes como copias del borde, evitando introducir ceros artificiales.

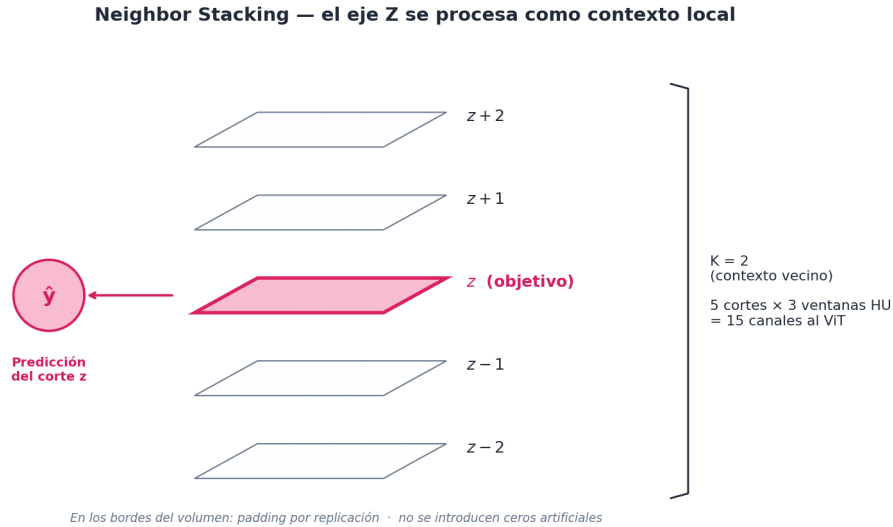


Figura 2: Esquema del **neighbor stacking**: para predecir el corte z se apilan sus vecinos $z - 2, \dots, z + 2$ y se procesan como un único tensor 2D con $5 \times 3 = 15$ canales. En los bordes del volumen el padding replica el corte más próximo en lugar de rellenar con ceros, evitando inyectar señales artificiales en la entrada del encoder.

La Figura 2 aclara cómo se traslada el contexto axial al encoder 2D: en lugar de pedirle al modelo que infiera la coherencia entre cortes a posteriori, se la entregamos directamente en la entrada. Cada predicción nace ya con información de profundidad sin pagar el coste de una convolución 3D.

5.2. Flujo de Tensores

El flujo principal es:

1. Entrada volumétrica: (B, 3, H, W, D).
2. Apilamiento de vecinos: (B*D, 15, H, W).
3. Encoder ViT 2D: extracción de características globales por corte.
4. Decoder convolucional 2D: recuperación de resolución espacial.
5. Recomposición volumétrica: (B, 2, H, W, D).

Así, el eje de profundidad se procesa como parte del batch interno. El modelo no hace convoluciones 3D, pero cada corte ve información de sus vecinos inmediatos. Es una solución intermedia: no captura dependencias largas en Z como una red 3D completa, pero sí reduce el efecto de “parpadeo” entre cortes que aparece en modelos 2D puros.

5.3. Encoder ViT Pre-entrenado

El encoder configurado es `vit_base_patch16_224.mae`, creado con `timm` (Wightman 2019). Este modelo fue pre-entrenado con Masked Autoencoders (He et al. 2022), una estrategia auto-supervisada que aprende representaciones visuales reconstruyendo regiones ocultas de la imagen.

El problema técnico es que ViT espera imágenes RGB de 3 canales, mientras que nuestro tensor tiene 15 canales. Para evitar reinicializar el primer `Conv2d` de `patch_embed`, se implementa una adaptación explícita:

1. Se carga un ViT pre-entrenado con `in_chans=3`.
2. Se promedian los pesos RGB del primer kernel.
3. Se replica el kernel promedio hasta 15 canales.
4. Se escala por $3 / 15$ para conservar la magnitud esperada de activación.

Con esto se preserva el conocimiento pre-entrenado en el encoder y se adapta la entrada médica multicanal sin romper la distribución inicial de activaciones.

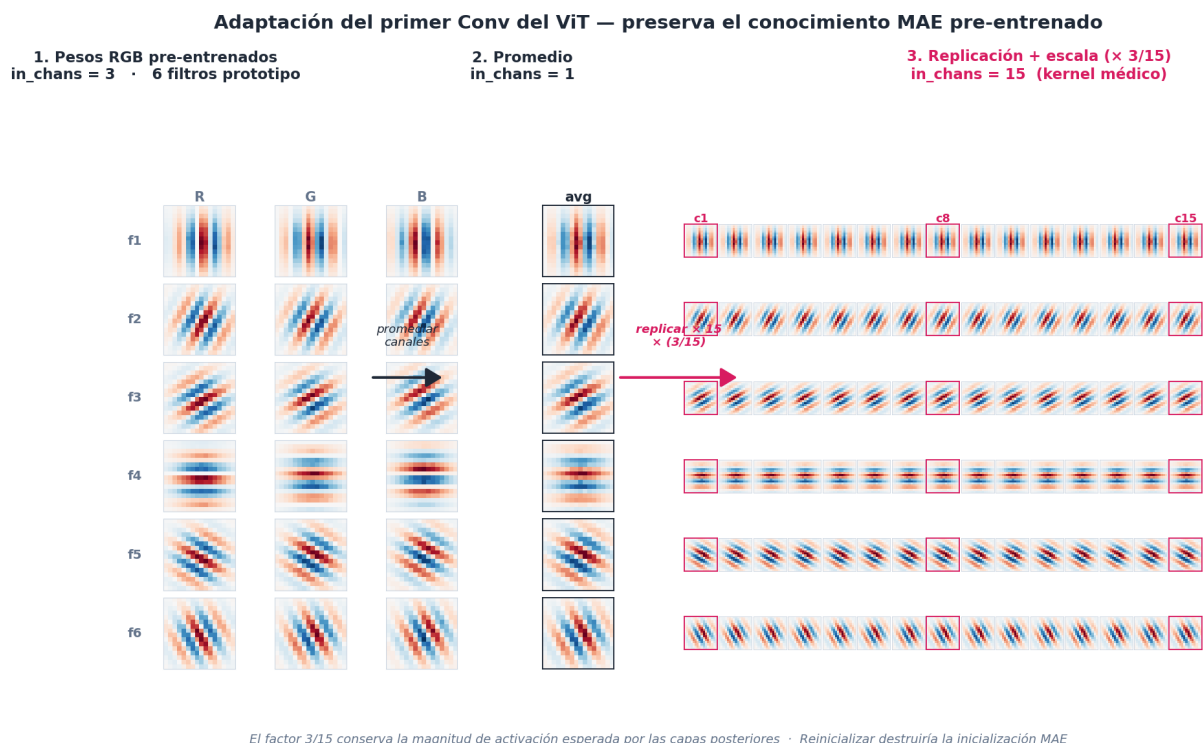


Figura 3: Adaptación del primer Conv2d de `patch_embed`. A la izquierda, los pesos RGB pre-entrenados con MAE (tres canales por filtro); en el centro, su promedio canal a canal; a la derecha, la replicación a 15 canales escalada por 3/15 para preservar la magnitud de activación esperada por las capas posteriores.

Como muestra la Figura 3, este truco evita reinicializar el primer kernel desde una distribución aleatoria y conserva los detectores de bordes y orientaciones que el ViT ya había aprendido sobre imágenes naturales. Reinicializar destruiría la inicialización MAE y obligaría a re-aprender desde cero filtros de bajo nivel que también son útiles en CT; el factor 3/15 es el ajuste de escala mínimo para que la varianza de activación tras el Conv siga en el mismo régimen que durante el pre-entrenamiento.

5.4. Decoder y Supervisión Profunda

El decoder consta de tres bloques `ConvTranspose2d` con factores de aumento 4x, 2x y 2x, sumando un 16x total compatible con el patch size del ViT. Además de la cabeza final a resolución completa, el modelo puede emitir cabezas auxiliares a 1/2 y 1/4 de resolución en XY.

Durante entrenamiento, la pérdida se calcula en las tres salidas con pesos:

$$1.0, \quad 0.5, \quad 0.25$$

La etiqueta se reduce únicamente en las dimensiones XY; el eje Z se conserva porque el modelo sigue prediciendo un mapa por corte. Esta **deep supervision** ayuda a que el gradiente llegue a capas tempranas del decoder y estabiliza el aprendizaje cuando el encoder está congelado.

5.5. Congelación y Descongelación

Por defecto, el encoder MAE permanece congelado (`unfreeze_epoch = -1`) y solo se entrenan el decoder y las cabezas de segmentación. Esto reduce memoria, acelera el entrenamiento y evita destruir representaciones útiles con pocos datos. El Makefile permite activar un descongelado controlado:


```
make phase4-fold FOLD=0 UNFREEZE_EPOCH=5 UNFREEZE_LR_FACTOR=0.1
```

Si se descongela, el learning rate del encoder se reduce mediante `UNFREEZE_LR_FACTOR`, evitando una actualización brusca de los pesos pre-entrenados.

6. Función de Pérdida

La segmentación tumoral está muy desbalanceada: la mayor parte del volumen es fondo, y el tumor puede ocupar una proporción muy pequeña. Una pérdida simétrica como cross-entropy tiende a favorecer soluciones conservadoras. Por ello se usa **Focal Tversky Loss**:

$$TI(P, G) = \frac{TP + \epsilon}{TP + \alpha FP + \beta FN + \epsilon}$$

$$FTL = (1 - TI)^\gamma$$

En la configuración activa:

- $\alpha = 0.3$
- $\beta = 0.7$
- $\gamma = 4/3$
- `include_background = false`

El parámetro β mayor que α penaliza más los falsos negativos que los falsos positivos. En oncología, perder parte de una lesión puede ser más problemático que sobredelimitar ligeramente el borde. El exponente focal desplaza atención hacia regiones difíciles, como bordes irregulares y zonas donde el tumor toca tejidos de densidad parecida.

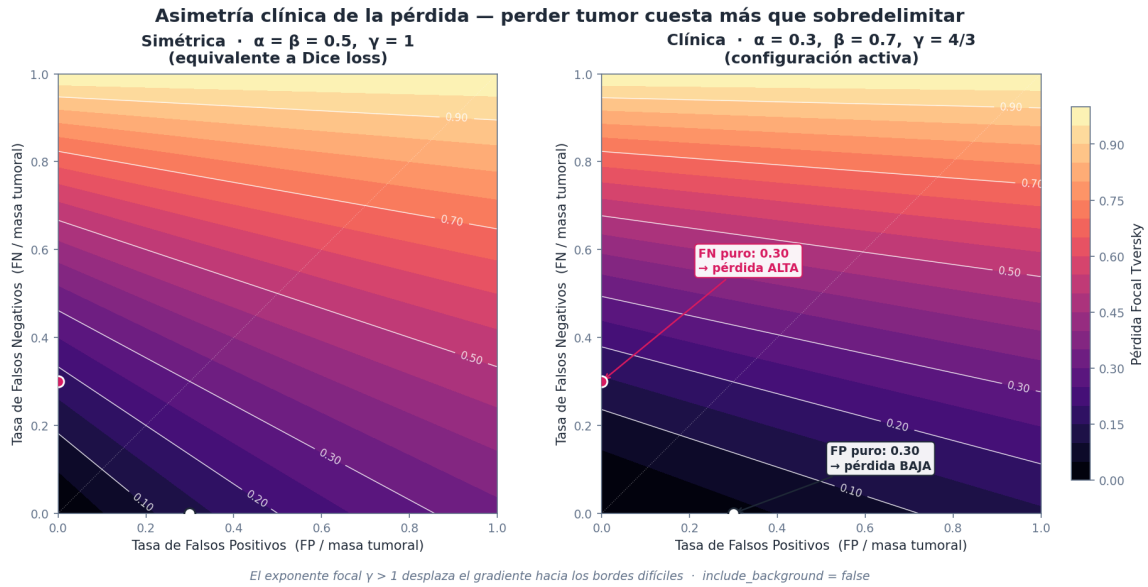


Figura 4: Paisaje de la pérdida Focal Tversky en el plano (FP, FN). Izquierda: configuración simétrica $\alpha = \beta = 0.5$, $\gamma = 1$ (equivalente a Dice loss). Derecha: configuración clínica activa $\alpha = 0.3$, $\beta = 0.7$, $\gamma = 4/3$, donde un mismo número de falsos negativos cuesta más que el mismo número de falsos positivos.

La Figura 4 deja explícita la asimetría que perseguimos: a igualdad de magnitud de error, perder tumor (FN) recibe un gradiente más intenso que sobredelimitarlo (FP), tal como se ve en la

diferencia de pendiente entre los dos ejes del panel derecho. El exponente focal $\gamma > 1$ además inclina el paisaje hacia las zonas difíciles, atenuando la actualización donde el modelo ya acierta y reforzándola en bordes mal segmentados.

7. Entrenamiento

7.1. Configuración Base

El proyecto usa Hydra para componer configuración desde `configs/`. La configuración por defecto combina:

- `data=task06`
- `model=vit_25d_lung`
- `training=local_5060`
- `experiment=phase4_full`

El perfil `local_5060` está ajustado para una GPU de 8 GB:

Parámetro	Valor
Patch	96 x 96 x 16
Batch size	1
Gradient accumulation	8
Optimizer	AdamW
LR	4e-4
Scheduler	Cosine warmup
Warmup	100 steps
AMP	activado
Encoder chunk size	32

El uso de acumulación de gradiente compensa el batch físico pequeño. A efectos de optimización, varios pasos forward/backward se agregan antes de actualizar pesos.

7.2. Validación por Volumen Completo

Aunque el entrenamiento se realiza con parches, la validación se calcula sobre volúmenes completos mediante `predict_volume`. Esto evita una discrepancia habitual: entrenar y medir en parches puede ocultar errores de continuidad, componentes falsos aislados o fallos en regiones alejadas del crop positivo.

Las métricas principales son:

- **Dice de foreground:** mide solapamiento entre predicción y máscara tumoral.
- **HD95:** distancia Hausdorff al percentil 95, más robusta que la Hausdorff máxima ante outliers.

Dos errores con Dice casi idéntico, HD95 muy distinto

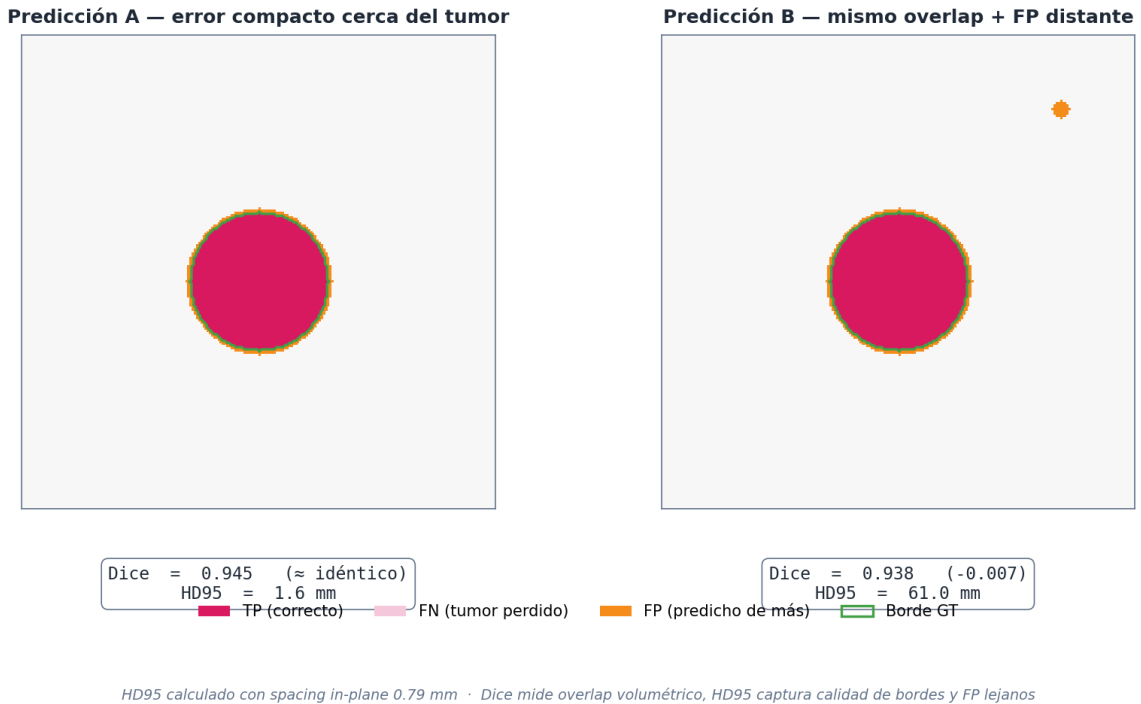


Figura 5: Dos predicciones con Dice prácticamente idéntico (0.945 vs 0.938) pero HD95 muy distinto (1.6 mm vs 61.0 mm). En **A** todo el error está concentrado cerca del tumor; en **B** aparece un pequeño falso positivo distante que apenas afecta al solapamiento volumétrico, pero degrada drásticamente la distancia de borde.

La Figura 5 explica por qué reportamos las dos métricas en paralelo: Dice es un solapamiento volumétrico robusto pero ciego a errores espacialmente lejanos, mientras que HD95 castiga precisamente ese tipo de fallo. En segmentación tumoral, un falso positivo aislado puede ser inofensivo (Dice apenas baja) o clínicamente relevante (HD95 se dispara), y solo combinando ambas métricas podemos distinguir un caso del otro.

El entrenador escribe `metrics.csv`, `summary.json`, `best.pt` y, si está habilitado, visualizaciones post-entrenamiento. También genera `metrics.png` con pérdida, Dice y HD95.

7.3. Checkpoints Ligeros

Una decisión práctica del proyecto es reducir la huella de los checkpoints. Con ViT-Base, guardar el encoder completo puede ocupar cientos de MB. Como el encoder congelado se reconstruye desde `timm`, el checkpoint por defecto guarda solo los parámetros entrenables y marca el estado como parcial. Al cargar para inferencia, se usa `strict=False` si el checkpoint declara `model_state_is_partial`.

Esto reduce `best.pt` de aproximadamente 348 MB a unos 12 MB cuando el encoder permanece congelado. Si se necesita reanudar entrenamiento completo, la configuración permite activar `save_resume_state=true` y `save_last_checkpoint=true`.

8. Inferencia Volumétrica

La inferencia se implementa en `src/lungseg/inference/sliding_window.py`. Para ViT25D, la estrategia automática no desliza en Z: mantiene la profundidad completa y desplaza ventanas

solo en XY. Esto conserva el contexto axial necesario para construir los vecinos de cada corte.

El flujo es:

1. Aplicar el mismo preprocesamiento de validación que en entrenamiento.
2. Dividir el plano XY en ventanas compatibles con el patch size del encoder.
3. Para cada ventana, construir el apilamiento de vecinos en Z.
4. Ejecutar el encoder por chunks de hasta `encoder_chunk_size=32` cortes.
5. Recomponer logits y tomar la clase con mayor probabilidad.

El chunking del encoder es clave. En validación, un volumen puede tener alrededor de 200 cortes; pasar todos como batch interno por ViT-Base puede saturar VRAM. Trocear el batch interno no cambia el resultado numérico, pero limita la memoria máxima.

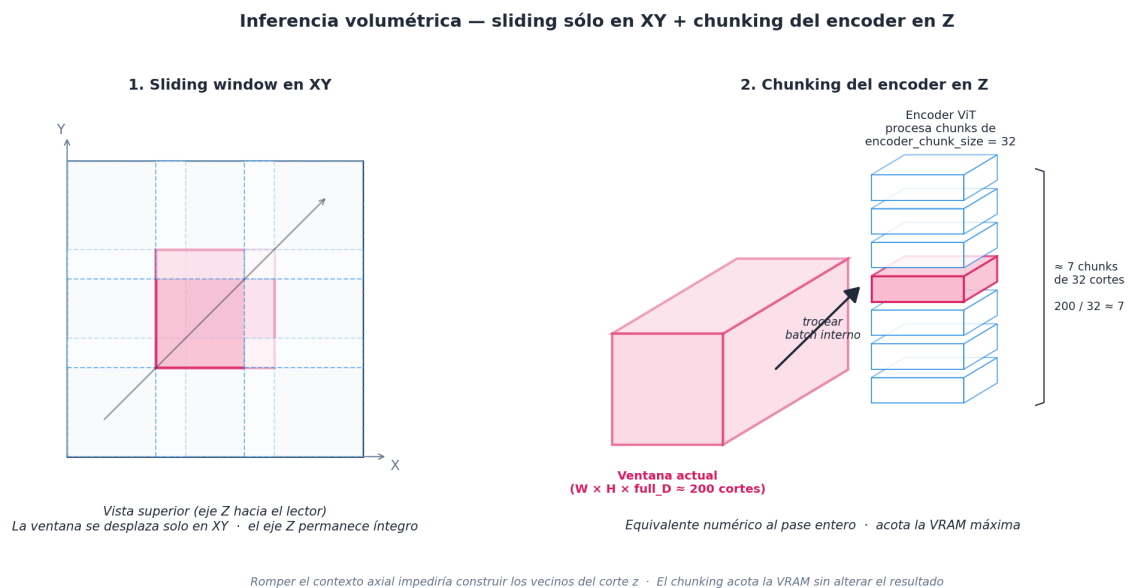


Figura 6: Inferencia volumétrica con ViT25D. Izquierda: la ventana deslizante recorre únicamente el plano XY, manteniendo la profundidad completa D para no romper el contexto axial. Derecha: el batch interno de cortes (~ 200) se trocea en chunks de hasta `encoder_chunk_size = 32` antes de pasar por el ViT, acotando el pico de VRAM sin alterar el resultado numérico.

La Figura 6 resume la diferencia frente a una sliding window 3D estándar: si deslizásemos también en Z, los primeros y últimos cortes de cada ventana no tendrían vecinos suficientes para construir el stack de cinco cortes, y el chunking del encoder dejaría de ser equivalente al forward completo. Mantener Z íntegro y trocear solo el batch interno preserva la equivalencia numérica con un forward sin chunks.

La CLI de predicción es:

```
python -m lungseg.cli predict \
  --checkpoint best.pt \
  --image volume.nii.gz \
  --output pred.nii.gz \
  --visualize
```

Si se proporciona una etiqueta, la visualización triplanar puede superponer predicción y ground truth.

9. Fase 6: Ablación

La fase de ablación estudia dos factores:

- **Fracción de datos:** 0.25, 0.5, 1.0.
- **Aumentación:** none frente a **standard**.

Cada celda se repite con tres semillas (0, 1, 2). Para que la comparación sea justa, el experimento usa un número fijo de iteraciones por celda. La reducción de datos se hace de forma estratificada por terciles de volumen tumoral, no eliminando pacientes al azar sin control.

El runner de ablación escribe un JSON por celda y `analysis.py` genera:

- `ablation_summary.csv`
- `ablation_violin.png`
- `REPORT_ABLATION.md`
- pruebas pareadas de Wilcoxon por fracción de datos

Los resultados actuales se leen automáticamente desde `outputs/` si existen.

Tabla 3

data_frac-	aug-		dice_me-	di-	di-	hd95_me-			
tion	ment_regi-		dian	ce_q1	ce_q3	dian	hd95_q1	hd95_q3	n
0.25	none		0.310391	0.270507	0.326149	64.6613	52.9073	67.7106	3
0.25	standard		0.309486	0.309032	0.341642	52.4284	51.8745	55.8707	3
0.5	none		0.370747	0.366984	0.386669	43.4311	43.0115	45.5188	3
0.5	standard		0.401377	0.400971	0.407754	33.3171	32.2715	35.0536	3
1	none		0.428246	0.423654	0.445945	34.317	29.4358	38.2536	3
1	standard		0.483236	0.471985	0.489389	28.8763	27.1705	30.2803	3

La tabla muestra un patrón coherente: con el 100 % de los datos, **standard** alcanza una mediana Dice superior a **none** y reduce HD95. Con el 50 % ocurre lo mismo, aunque con menor margen. En el 25 %, la mediana Dice es muy similar entre regímenes, pero HD95 mejora al usar aumentación estándar, lo que sugiere bordes menos erráticos incluso cuando el solapamiento global cambia poco.

La interpretación estadística debe ser prudente. Con solo tres semillas por celda, el test de Wilcoxon tiene potencia limitada; un `p_value` no significativo no implica ausencia de efecto, sino que no hay evidencia suficiente con ese tamaño muestral. En este contexto, la ablación nos sirve más para observar tendencias y estabilidad que para cerrar una conclusión estadística fuerte.

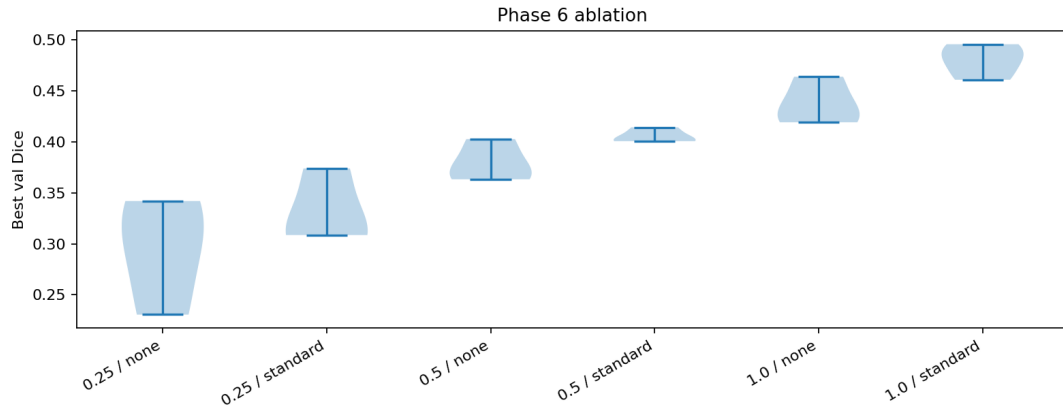


Figura 7: Distribución del Dice Score por fracción de datos y régimen de aumentación.

10. Resultados Cualitativos

10.1. Curvas de Entrenamiento

La Figura 8 resume la dinámica esperada del entrenamiento: reducción progresiva de Focal Tversky, aumento del Dice de validación y descenso de HD95. En el pipeline real, esta figura se genera a partir de `metrics.csv` al finalizar `Trainer.fit()`.

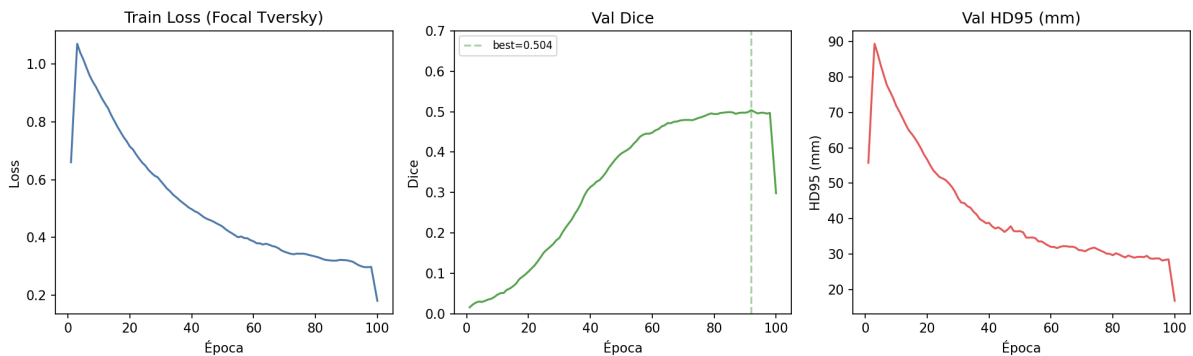


Figura 8: Curvas de entrenamiento y validación: pérdida Focal Tversky, Dice y HD95.

El objetivo de monitorizar Dice y HD95 simultáneamente es evitar una lectura incompleta. Dos máscaras pueden tener Dice parecido y distancias de borde muy distintas. En segmentación tumoral, una mejora de HD95 suele indicar contornos más estables y menos falsos positivos alejados.

10.2. Visualización Triplanar

La visualización triplanar permite inspeccionar la predicción en axial, coronal y sagital. Es especialmente útil para detectar errores que no se ven en un solo corte: discontinuidades, fugas hacia mediastino o máscaras que se desplazan respecto al tumor.

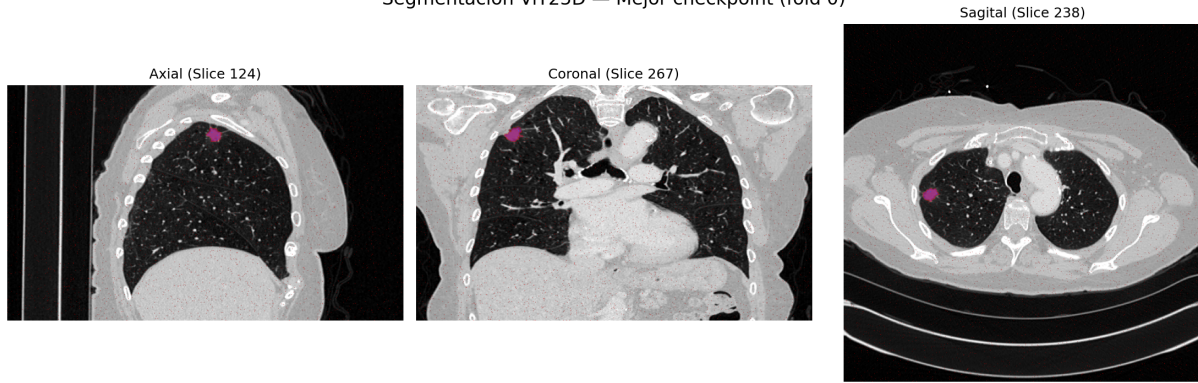


Figura 9: Visualización triplanar centrada en la lesión.

En una evaluación completa, esta figura debe revisarse junto a la métrica por caso. Un Dice medio puede ocultar que el modelo funciona bien en masas grandes pero falla en lesiones pequeñas. La inspección cualitativa ayuda a localizar esos patrones.

10.3. Mosaico Axial

El mosaico axial muestra varios cortes consecutivos alrededor de la lesión. Esta vista aproxima el desplazamiento que haría un radiólogo al recorrer el volumen en un PACS.

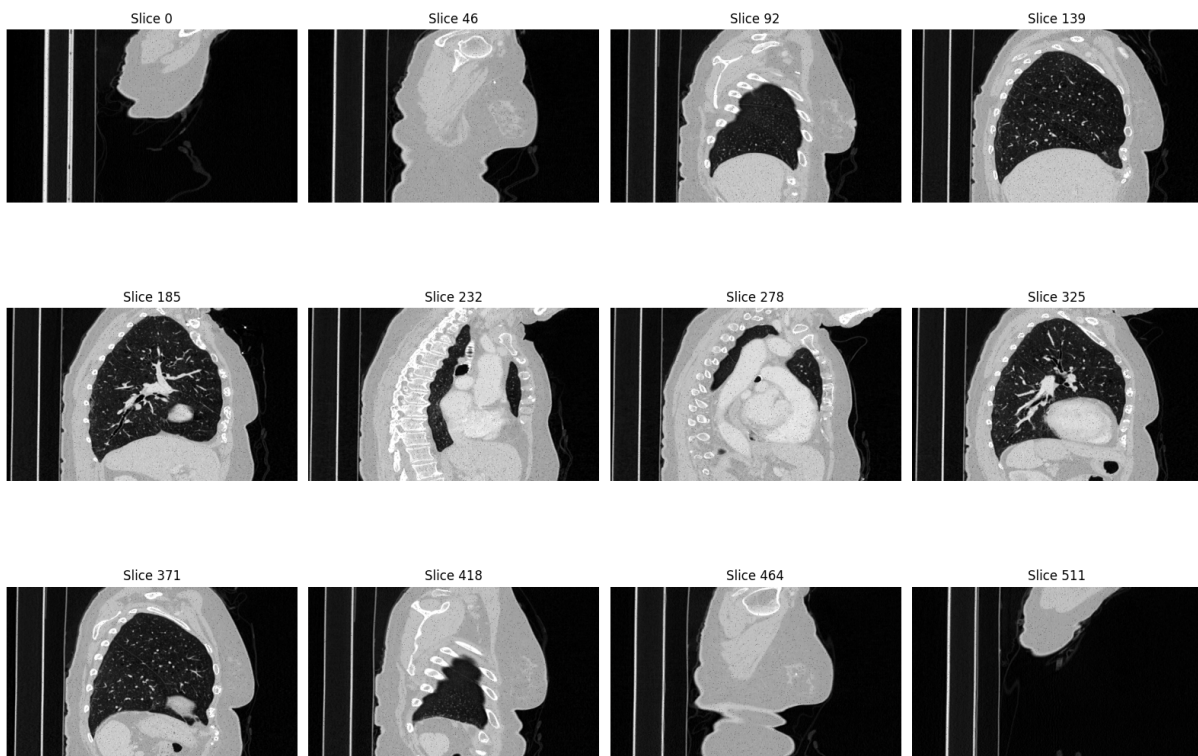


Figura 10: Mosaico de cortes axiales con máscara de segmentación.

El principal criterio visual es la continuidad: la máscara debería crecer y decrecer de forma suave a lo largo de Z. El neighbor stacking está diseñado precisamente para reducir segmentaciones intermitentes corte a corte.

i Trazabilidad de los artefactos

Para ser transparentes, el repositorio incluye `scripts/generate_memoria_outputs.py` para generar los archivos que usa esta memoria cuando no se dispone de una ejecución larga completa. Las visualizaciones emplean un CT real del dataset y máscaras de apoyo; las curvas y la ablación generadas por ese script son artefactos reproducibles para documentación, no una evaluación ciega oficial. El flujo de entrenamiento y ablación escribe los mismos tipos de archivos cuando se ejecuta sobre runs reales.

11. Ingeniería y Reproducibilidad

11.1. Estructura del Paquete

El código principal vive en `src/lungseg/`:

- `data/`: splits, transformaciones, dataloaders y máscaras pulmonares.
- `models/`: definición de ViT25D.
- `training/`: entrenador, pérdidas y schedulers.
- `inference/`: inferencia por sliding window y postprocesado.
- `ablation/`: ejecución y análisis de Fase 6.
- `radiomics/` y `classification/`: extensión opcional para LIDC-IDRI.
- `utils/`: métricas, logging, visualización y determinismo.

La configuración está separada en `configs/`, lo que permite cambiar hardware, modelo o experimento sin modificar código. El Makefile envuelve los comandos frecuentes y documenta las rutas recomendadas.

11.2. Comandos Principales

El flujo mínimo desde cero es:

```
make bootstrap
make splits
make precompute-masks
make phase4-fold FOLD=0
```

Para ejecutar la ablación:

```
make phase6
```

Y para validar el entorno:

```
make doctor
make qa
```

Esta capa de automatización reduce errores manuales y facilita repetir la misma configuración en local, Kaggle o Colab.

11.3. Tests

La suite de `pytest` cubre los contratos más críticos:

- `test_vit_25d.py`: shapes, deep supervision, congelación del encoder, adaptación de pesos y equivalencia del chunking.
- `test_neighbor_stack.py`: construcción correcta de vecinos y padding por replicación.

- `test_multi_window.py` y `test_transforms.py`: ventanas HU, no mutación de MetaTensor, orden de transforms y prohibición de flip izquierda-derecha.
- `test_lung_mask_transform.py`: aplicación de máscara pulmonar y errores claros si falta caché.
- `test_splits.py`: ausencia de solapamiento entre pacientes, determinismo y cobertura de casos.
- `test_phase5_phase6.py`: contratos de ablación y fallo explícito de clasificación sobre Task06.

Estos tests no sustituyen una validación médica, pero sí protegen decisiones de diseño que podrían romperse fácilmente durante refactors.

12. Discusión

El enfoque 2.5D nos parece un compromiso razonable entre contexto volumétrico y coste computacional. Frente a una red 2D pura, cada predicción incorpora información axial cercana. Frente a una red 3D pura, el modelo puede usar un encoder ViT pre-entrenado y reducir la presión de VRAM.

La ventaja más clara es práctica: nos permite experimentar con ViT-Base en hardware accesible. El encoder MAE aporta representaciones ya aprendidas y el decoder se adapta a la tarea médica con relativamente pocos parámetros entrenables. Esta estrategia encaja bien con un dataset pequeño como Task06, aunque no elimina la dificultad del problema.

Sin embargo, el diseño también tiene limitaciones:

- El contexto en Z es local. Dependencias largas, como la continuidad completa de una masa grande, solo se capturan de forma indirecta.
- El prior pulmonar puede ser contraproducente si el tumor invade o contacta regiones que `lungmask` excluye parcialmente.
- La validación interna con 63 casos tiene alta incertidumbre; los resultados deben reportarse por fold y con intervalos o semillas.
- Las métricas globales no capturan todos los errores clínicamente relevantes. Un pequeño falso negativo en borde tumoral puede importar más que lo que refleja el Dice.
- La comparación con arquitecturas 3D no está cerrada. Para afirmar superioridad haría falta entrenar baselines fuertes como nnU-Net (Isensee et al. 2021) bajo el mismo protocolo.

El aprendizaje principal es que la ingeniería del flujo importa tanto como la arquitectura. Splits por paciente, manejo correcto de spacing, ventanas HU, caché no mutante, validación por volumen completo y checkpoints reproducibles son condiciones necesarias para que la métrica sea interpretable.

13. Conclusiones

Este proyecto implementa un segmentador pulmonar 2.5D basado en ViT pre-entrenado y lo integra en un flujo MONAI reproducible. La solución combina cinco ideas principales:

1. Contexto axial mediante neighbor stacking.
2. Tres ventanas HU como canales complementarios.
3. Máscara pulmonar como prior anatómico.
4. Transferencia MAE adaptada a 15 canales.
5. Pérdida Focal Tversky con penalización reforzada de falsos negativos.

La Fase 6 añade una capa de análisis experimental al estudiar cómo cambian Dice y HD95 con

menos datos y con o sin aumentación. Aunque los resultados deben interpretarse con cautela por el tamaño de muestra, la infraestructura queda preparada para sustituir artefactos de memoria por ejecuciones completas en todos los folds.

13.1. Validez del Proyecto Bajo Restricciones de Hardware

Aunque los resultados cuantitativos no son altos, consideramos que el proyecto sigue siendo válido desde un punto de vista académico y empírico. No hemos demostrado que el modelo sea clínicamente competitivo, pero sí hemos construido una prueba de concepto completa: hay una hipótesis técnica clara, una implementación funcional, particiones por paciente, métricas volumétricas, visualizaciones, ablación y tests sobre partes importantes del código.

Las restricciones de hardware han condicionado bastante el alcance experimental. Para poder entrenar en una GPU limitada tuvimos que reducir tamaño de parches, batch efectivo, número de iteraciones, número de semillas y exploración de hiperparámetros. También se mantuvo el encoder congelado por defecto y se usaron checkpoints parciales para ahorrar disco. Estas decisiones hacen que los resultados actuales no deban leerse como el techo real de la arquitectura, sino como una evaluación inicial bajo condiciones bastante ajustadas.

Con mejor hardware no se puede prometer automáticamente un resultado excelente, porque el dataset sigue siendo pequeño y la tarea es difícil. Aun así, sí sería razonable esperar una evaluación más justa y probablemente mejores resultados: se podrían entrenar más épocas, ejecutar todos los folds, probar más semillas, usar parches mayores, descongelar parcialmente el encoder, ajustar mejor la aumentación y comparar contra baselines fuertes como nnU-Net. Por tanto, la conclusión más prudente es que el proyecto tiene bases sólidas, pero necesita más capacidad de cómputo y una validación más amplia para medir su potencial real.

Como trabajo futuro, las líneas más naturales son: ejecutar una validación completa multi-fold, añadir intervalos de confianza por bootstrap, comparar contra un baseline nnU-Net, estudiar postprocesado de componentes conectados y evaluar si descongelar parcialmente el encoder mejora resultados sin sobreajuste.

14. Bibliografía

- Abraham, Nabila, y Naimul M Khan. 2019. «A Novel Focal Tversky Loss Function with Improved Attention U-Net for Lesion Segmentation». En *2019 IEEE 16th International Symposium on Biomedical Imaging (ISBI)*, 683-87. IEEE.
- Antonelli, Michela, Annika Reinke, Spyridon Bakas, et al. 2022. «The Medical Segmentation Decathlon». *Nature Communications* 13: 4128. <https://doi.org/10.1038/s41467-022-30695-9>.
- Dosovitskiy, Alexey et al. 2021. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale». En *International Conference on Learning Representations (ICLR)*.
- He, Kaiming, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, y Ross Girshick. 2022. «Masked Autoencoders Are Scalable Vision Learners». En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16000-16009.
- Hofmanninger, Johannes et al. 2020. «Automatic lung segmentation in routine imaging is a solved problem». *arXiv:2001.11767*.
- Isensee, Fabian et al. 2021. «nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation». *Nature Methods* 18: 203-11.
- Project MONAI. 2020. «MONAI: Medical Open Network for AI». <https://monai.io>.
- Simpson, Amber L et al. 2019. «A large annotated medical image dataset for the development and evaluation of segmentation algorithms». *arXiv:1902.09063*.
- Wightman, Ross. 2019. «PyTorch Image Models». *GitHub repository*.